

Objektinis programavimas 2

Generated by Doxygen 1.17.0

1 Antras_Laboratorinis	1
1.1 Programos aprašymas	1
1.2 Versija - v1.1	2
1.3 Versija - v1.2	2
1.3.0.1 Naudoti metodai ir jų paskirtys	3
1.4 Versija - v1.5	3
1.5 Versija - v2.0	3
1.6 Kompiuterio charakteristikos	3
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 RandInt Class Reference	11
5.1.1 Constructor & Destructor Documentation	11
5.1.1.1 RandInt()	11
5.1.2 Member Function Documentation	11
5.1.2.1 operator()()	11
5.1.3 Member Data Documentation	12
5.1.3.1 dist	12
5.1.3.2 mt	12
5.1.3.3 rd	12
5.2 Studentas Class Reference	12
5.2.1 Constructor & Destructor Documentation	13
5.2.1.1 Studentas() [1/5]	13
5.2.1.2 Studentas() [2/5]	13
5.2.1.3 Studentas() [3/5]	14
5.2.1.4 Studentas() [4/5]	14
5.2.1.5 Studentas() [5/5]	14
5.2.1.6 ~Studentas()	14
5.2.2 Member Function Documentation	14
5.2.2.1 apskaiciuot_vidurkis()	14
5.2.2.2 apskaiciuoti_mediana()	14
5.2.2.3 Clear()	14
5.2.2.4 egzaminas()	14
5.2.2.5 isClear()	14
5.2.2.6 mediana()	15
5.2.2.7 operator=() [1/2]	15

5.2.2.8 operator=() [2/2]	15
5.2.2.9 pazymiai()	15
5.2.2.10 printInfo()	15
5.2.2.11 setEgzaminas()	15
5.2.2.12 setGalutinisM()	15
5.2.2.13 setGalutinisV()	15
5.2.2.14 setPazymiai()	15
5.2.2.15 vidurkis()	16
5.2.3 Friends And Related Symbol Documentation	16
5.2.3.1 operator<<	16
5.2.3.2 operator==	16
5.2.3.3 operator>>	16
5.2.4 Member Data Documentation	16
5.2.4.1 egz_paz_	16
5.2.4.2 Mediana_	16
5.2.4.3 paz_	16
5.2.4.4 Vidurkis_	16
5.3 Zmogus Class Reference	17
5.3.1 Constructor & Destructor Documentation	17
5.3.1.1 Zmogus() [1/4]	17
5.3.1.2 Zmogus() [2/4]	17
5.3.1.3 Zmogus() [3/4]	17
5.3.1.4 Zmogus() [4/4]	18
5.3.1.5 ~Zmogus()	18
5.3.2 Member Function Documentation	18
5.3.2.1 operator=() [1/2]	18
5.3.2.2 operator=() [2/2]	18
5.3.2.3 pavarde()	18
5.3.2.4 printInfo()	18
5.3.2.5 setPavarde()	18
5.3.2.6 setVardas()	18
5.3.2.7 vardas()	18
5.3.3 Member Data Documentation	19
5.3.3.1 Pavarde_	19
5.3.3.2 Vardas_	19
6 File Documentation	21
6.1 catch_testai.cpp File Reference	21
6.1.1 Macro Definition Documentation	21
6.1.1.1 CATCH_CONFIG_MAIN	21
6.1.2 Function Documentation	21
6.1.2.1 TEST_CASE()	21

6.2 funkcijos.cpp File Reference	22
6.2.1 Function Documentation	22
6.2.1.1 duomenu_ivedimas()	22
6.2.1.2 failu_generavimas()	22
6.2.1.3 isvedimas_i_du_failus()	22
6.2.1.4 mediana()	23
6.2.1.5 meniu()	23
6.2.1.6 operator<<()	23
6.2.1.7 operator>>()	23
6.2.1.8 pagal_med_did()	23
6.2.1.9 pagal_med_maz()	23
6.2.1.10 pagal_pavarde()	23
6.2.1.11 pagal_varda()	23
6.2.1.12 pagal_vid_did()	24
6.2.1.13 pagal_vid_maz()	24
6.2.1.14 pasirinkimas()	24
6.2.1.15 rezultatai()	24
6.2.1.16 rezultatu_isvedimas()	24
6.2.1.17 rikiavimas()	24
6.2.1.18 skaiciu_generevimas()	24
6.2.1.19 skaiciuoti_viska()	24
6.2.1.20 skaityti_faila()	24
6.2.1.21 skaityti_faila_automatiskai()	25
6.2.1.22 studentu_skirstymas()	25
6.2.1.23 vidurkis()	25
6.3 konteineriai.h File Reference	25
6.3.1 Function Documentation	26
6.3.1.1 antra_strategija()	26
6.3.1.2 isvedimas()	26
6.3.1.3 pirma_strategija()	26
6.3.1.4 rusiavimas_maz()	26
6.3.1.5 skaityti_failus()	26
6.3.1.6 strategijos_du_tyrimas()	26
6.3.1.7 strategijos_trys_tyrimas()	26
6.3.1.8 strategiju_tyrimas()	27
6.3.1.9 stunedu_skirstymas()	27
6.3.1.10 trecia_strategija()	27
6.3.1.11 tyrimas()	27
6.4 konteineriai.h	27
6.5 konteinerius_tyrimas.cpp File Reference	31
6.5.1 Function Documentation	31
6.5.1.1 main()	31

6.6 RandInt.hpp File Reference	31
6.7 RandInt.hpp	31
6.8 README.md File Reference	31
6.9 strategija_1.cpp File Reference	31
6.9.1 Function Documentation	32
6.9.1.1 main()	32
6.10 strategija_2.cpp File Reference	32
6.10.1 Function Documentation	32
6.10.1.1 main()	32
6.11 strategija_3.cpp File Reference	32
6.11.1 Function Documentation	32
6.11.1.1 main()	32
6.12 studentas.h File Reference	33
6.12.1 Function Documentation	33
6.12.1.1 duomenu_ivedimas()	33
6.12.1.2 failu_generavimas()	34
6.12.1.3 isvedimas_i_du_failus()	34
6.12.1.4 meniu()	34
6.12.1.5 pagal_med_did()	34
6.12.1.6 pagal_med_maz()	34
6.12.1.7 pagal_pavarde()	34
6.12.1.8 pagal_varda()	34
6.12.1.9 pagal_vid_did()	34
6.12.1.10 pagal_vid_maz()	35
6.12.1.11 pasirinkimas()	35
6.12.1.12 rezultatai()	35
6.12.1.13 rezultatu_isvedimas()	35
6.12.1.14 rikiavimas()	35
6.12.1.15 skaiciu_generevimas()	35
6.12.1.16 skaiciuoti_viska()	35
6.12.1.17 skaityti_faila()	35
6.12.1.18 skaityti_faila_automatiskai()	35
6.12.1.19 studentu_skirstymas()	36
6.13 studentas.h	36
6.14 vector.cpp File Reference	38
6.14.1 Function Documentation	38
6.14.1.1 main()	38
6.15 zmogus.h File Reference	38
6.16 zmogus.h	39

Chapter 1

Antras_Laboratorinis

1.1 Programos aprašymas

Ši programa yra skirta apdoroti studentų duomenis ir jų akademinius pasiekimus. **Pagrindinės programos funkcijos**

- 1. Įvesti visus studento duomenis ranka;
- 2. Sugeneruoti atsitiktinius pažymius, bet studento vardą ir pavardę įvesti ranka;
- 3. Sudenetuoti studento vardą, pavardę ir pažymius;
- 4. Nuskaityti duomenis iš failo;
- 5. Testuoti programą;

Taip pat programoje yra trys strategijos, kurios skirtos parodyti kaip kinta programos veikimo laikas nuo naudojamų konteinerių (vector, list, deque) duomenų skirstymo į dvi grupes (pažangių ir nepažangių) metu.

Pirmoje strategijoje skirstymas vyksta į du to paties tipo konteinerius, antrosios strategijos metu sukuriamas tik vienas papildomas konteineris ir į jį perkeliami nepažangūs mokiniai, o pažangūs lieka pradiniam. Trečioji strategija yra optimizuota antroji strategija naudojant Standard Template Library (STL)

Atlikus tyrimus galime pamatyti, jog greičiausia yra trečioji strategija, naudojant vector.

Naudojimosi instrukcija

1. Susiinstaliuoti **make** (Windows naudotojams rekomenduojama per [GNUWin32](#))
2. Atsidaryti terminalą
3. Klonuoti programos repozitoriją:

```
git clone https://github.com/vaivaeva201/objektinis\_programavimas\_2
```

4. **Užėti į projekto aplanką:**

```
cd objektinis_programavimas_1
```

5. Paleisti norimą funkciją:

- `make main`
- pagrindinė programa
- `make run0`
- pradinio tyrimo paleidimas
- `make run1`
- 1 strategijos testavimas
- `make run2`
- 2 strategijos testavimas
- `make run3`
- 3 strategijos testavimas
- `make unit`
- unit testai

1.2 Versija - v1.1

Šakoje v1.1 lyginome "class" ir "struct" naudojimą apdorojant studentų duomenis. v1.1 versija pritaikyta "class" ir buvo lyginama su v1.0 versijos "struct". Buvo naudojamas vektor konteineris su 3 strategija. Pateikti rezultatai yra trijų iteracijų vidurkis.

Tipas	Failo dydis	Optimizavimo vėliava	.exe failo dydis	Programos veikimo laikas
CLASS	100000	-	239K	2.48582 s
CLASS	1000000	-	239K	25.032 s
STRUCT	100000	-	238K	1.93843 s
STRUCT	1000000	-	238K	19.6844 s
CLASS	100000	-O1	121K	0.84203 s
CLASS	1000000	-O1	121K	8.11251 s
STRUCT	100000	-O1	120K	0.714521 s
STRUCT	1000000	-O1	120K	7.23889 s
CLASS	100000	-O2	121K	0.816232 s
CLASS	1000000	-O2	121K	7.9286 s
STRUCT	100000	-O2	120K	0.672481 s
STRUCT	1000000	-O2	120K	6.98227 s
CLASS	100000	-O3	121K	0.792985 s
CLASS	1000000	-O3	121K	7.81915 s
STRUCT	100000	-O3	120K	0.668681 s
STRUCT	1000000	-O3	120K	6.9911 s

1.3 Versija - v1.2

Šioje versijoje klasė '[Studentas](#)' buvo patobulinta pritaikant "Rule of Five" ir perdengtus išvesties ir įvesties metodus.

1.3.0.1 Naudoti metodai ir jų paskirtys

1.3.0.1.1 Rule of Five

- Kopijavimo konstruktorius - sukuria tikslią egzistuojančio objekto reikšmių kopiją;
- Kopijavimo priskyrimo operatorius - jau inicializuotam objektui priskiriama nauja reikšmė iš kito egzistuojančio objekto;
- Move konstruktorius - perkelia vieno objekto reikšmes į kitą objektą ir ištrina pirmąjį objektą;
- Move priskyrimo operatorius - perkialia vieno objekto reikšmes į jau inicializuotą default objektą ir ištrina pirmojo objektą.
- Destruktorius - atsakingas už objektui priskirtų reikšmių išvalymą;

1.3.0.1.2 Perdengti įvesties ir išvesties metodai

- Įvesties operatorius - leidžia nuskaityti vieno studento visus duomenis tiesiai iš pasirinkto srauto;
- Išvesties operatorius - leidžia į pasirinktą srautą išvesti vieno studento duomenis;

1.4 Versija - v1.5

Šioje versijoje pridėt nauja klasė '[Zmogus](#)', ši klasė yra abstrakti. Dabar klasė '[Studentas](#)' paveldi klasę '[Zmogus](#)', atitinkamai pakeisti konstruktoriai ir destruktoriai.

Kadangi klasė '[Zmogus](#)' yra abstrakti negalima sukurti tokio tipo objekto:

- Patikrinti klases buvo naudoti tie patys testai kaip ir v1.2 versijoje:

1.5 Versija - v2.0

v2.0 versijoje sukurta dokumentacija ir atlikti Unit testai. Sukurti testam buvo naudojama Catch2 testavimo framework.

1.6 Kompiuterio charakteristikos

Testuojamos sistemos parametrai

Parametras	Reikšmė
CPU	Apple M1, 8 core
RAM	8 GB
SSD	256 GB

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

RandInt	11
Zmogus	17
Studentas	12

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

RandInt		11
Studentas		12
Zmogus		17

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

catch_testai.cpp	21
funkcijos.cpp	22
konteineriai.h	25
konteinerius_tyrimas.cpp	31
RandInt.hpp	31
strategija_1.cpp	31
strategija_2.cpp	32
strategija_3.cpp	32
studentas.h	33
vector.cpp	38
zmogus.h	38

Chapter 5

Class Documentation

5.1 RandInt Class Reference

```
#include <RandInt.hpp>
```

Public Member Functions

- [RandInt](#) (int low, int high)
- int [operator\(\)](#) ()

Private Attributes

- std::random_device [rd](#)
- std::mt19937 [mt](#)
- std::uniform_int_distribution< int > [dist](#)

5.1.1 Constructor & Destructor Documentation

5.1.1.1 RandInt()

```
RandInt::RandInt (  
    int low,  
    int high) [inline]
```

5.1.2 Member Function Documentation

5.1.2.1 operator()

```
int RandInt::operator() () [inline]
```

5.1.3 Member Data Documentation

5.1.3.1 dist

```
std::uniform_int_distribution<int> RandInt::dist [private]
```

5.1.3.2 mt

```
std::mt19937 RandInt::mt [private]
```

5.1.3.3 rd

```
std::random_device RandInt::rd [private]
```

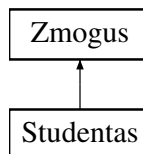
The documentation for this class was generated from the following file:

- [RandInt.hpp](#)

5.2 Studentas Class Reference

```
#include <studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (string v, string p, vector< int > pazymiai)
- [Studentas](#) (std::istream &is)
- [Studentas](#) (const [Studentas](#) &s)
- [Studentas](#) ([Studentas](#) &&s) noexcept
- [~Studentas](#) ()
- const vector< int > & [pazymiai](#) () const
- int [egzaminas](#) () const
- double [vidurkis](#) () const
- double [mediana](#) () const
- void [setGalutinisV](#) (double galutVid)
- void [setGalutinisM](#) (double galutMed)
- void [setEgzaminas](#) (int egz)
- void [setPazymiai](#) (vector< int > paz)
- void [apskaiciuot_vidurkis](#) ()
- void [apskaiciuoti_mediana](#) ()
- [Studentas](#) & [operator=](#) (const [Studentas](#) &s)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&s) noexcept
- void [Clear](#) ()
- bool [isClear](#) ()
- void [printInfo](#) () const override

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (string v, string p)
- [Zmogus](#) (const [Zmogus](#) &z)
- [Zmogus](#) ([Zmogus](#) &&z) noexcept
- string [vardas](#) () const
- string [pavarde](#) () const
- void [setVardas](#) (string var)
- void [setPavarde](#) (string pav)
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &z)
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&z) noexcept
- virtual [~Zmogus](#) ()

Private Attributes

- vector< int > [paz_](#)
- int [egz_paz_](#)
- double [Vidurkis_](#)
- double [Mediana_](#)

Friends

- bool [operator==](#) (const [Studentas](#) &s1, const [Studentas](#) &s2)
- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [Vardas_](#)
- string [Pavarde_](#)

5.2.1 Constructor & Destructor Documentation

5.2.1.1 [Studentas\(\)](#) [1/5]

```
Studentas::Studentas () [inline]
```

5.2.1.2 [Studentas\(\)](#) [2/5]

```
Studentas::Studentas (
    string v,
    string p,
    vector< int > pazymiai) [inline]
```

5.2.1.3 Studentas() [3/5]

```
Studentas::Studentas (  
    std::istream & is)
```

5.2.1.4 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & s) [inline]
```

5.2.1.5 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && s) [inline], [noexcept]
```

5.2.1.6 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

5.2 Member Function Documentation

5.2.2.1 apskaiciuot_vidurkis()

```
void Studentas::apskaiciuot_vidurkis ()
```

5.2.2.2 apskaiciuoti_mediana()

```
void Studentas::apskaiciuoti_mediana ()
```

5.2.2.3 Clear()

```
void Studentas::Clear () [inline]
```

5.2.2.4 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

5.2.2.5 isClear()

```
bool Studentas::isClear () [inline]
```

5.2.2.6 mediana()

```
double Studentas::mediana () const [inline]
```

5.2.2.7 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & s) [inline]
```

5.2.2.8 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && s) [inline], [noexcept]
```

5.2.2.9 pazymiai()

```
const vector< int > & Studentas::pazymiai () const [inline]
```

5.2.2.10 printInfo()

```
void Studentas::printInfo () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.2.2.11 setEgzaminas()

```
void Studentas::setEgzaminas (  
    int egz) [inline]
```

5.2.2.12 setGalutinisM()

```
void Studentas::setGalutinisM (  
    double galutMed) [inline]
```

5.2.2.13 setGalutinisV()

```
void Studentas::setGalutinisV (  
    double galutVid) [inline]
```

5.2.2.14 setPazymiai()

```
void Studentas::setPazymiai (  
    vector< int > paz) [inline]
```

5.2.2.15 vidurkis()

```
double Studentas::vidurkis () const [inline]
```

5.2.3 Friends And Related Symbol Documentation

5.2.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Studentas & s) [friend]
```

5.2.3.2 operator==

```
bool operator== (  
    const Studentas & s1,  
    const Studentas & s2) [friend]
```

5.2.3.3 operator>>

```
std::istream & operator>> (  
    std::istream & in,  
    Studentas & s) [friend]
```

5.2.4 Member Data Documentation

5.2.4.1 egz_paz_

```
int Studentas::egz_paz_ [private]
```

5.2.4.2 Mediana_

```
double Studentas::Mediana_ [private]
```

5.2.4.3 paz_

```
vector<int> Studentas::paz_ [private]
```

5.2.4.4 Vidurkis_

```
double Studentas::Vidurkis_ [private]
```

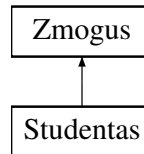
The documentation for this class was generated from the following file:

- [studentas.h](#)

5.3 Zmogus Class Reference

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (string v, string p)
- [Zmogus](#) (const [Zmogus](#) &z)
- [Zmogus](#) ([Zmogus](#) &&z) noexcept
- string [vardas](#) () const
- string [pavarde](#) () const
- void [setVardas](#) (string var)
- void [setPavarde](#) (string pav)
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &z)
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&z) noexcept
- virtual [~Zmogus](#) ()
- virtual void [printInfo](#) () const =0

Protected Attributes

- string [Vardas_](#)
- string [Pavarde_](#)

5.3.1 Constructor & Destructor Documentation

5.3.1.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

5.3.1.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    string v,
    string p) [inline]
```

5.3.1.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & z) [inline]
```

5.3.1.4 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && z) [inline], [noexcept]
```

5.3.1.5 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

5.3.2 Member Function Documentation

5.3.2.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (  
    const Zmogus & z) [inline]
```

5.3.2.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (  
    Zmogus && z) [inline], [noexcept]
```

5.3.2.3 pavarde()

```
string Zmogus::pavarde () const [inline]
```

5.3.2.4 printInfo()

```
virtual void Zmogus::printInfo () const [pure virtual]
```

Implemented in [Studentas](#).

5.3.2.5 setPavarde()

```
void Zmogus::setPavarde (  
    string pav) [inline]
```

5.3.2.6 setVardas()

```
void Zmogus::setVardas (  
    string var) [inline]
```

5.3.2.7 vardas()

```
string Zmogus::vardas () const [inline]
```

5.3.3 Member Data Documentation

5.3.3.1 Pavarde_

```
string Zmogus::Pavarde_ [protected]
```

5.3.3.2 Vardas_

```
string Zmogus::Vardas_ [protected]
```

The documentation for this class was generated from the following file:

- [zmogus.h](#)

Chapter 6

File Documentation

6.1 catch_testai.cpp File Reference

```
#include <catch2/catch_all.hpp>
#include "studentas.h"
```

Macros

- `#define` [CATCH_CONFIG_MAIN](#)

Functions

- [TEST_CASE](#) ("Studento klasės 'Rule of five' testai", "[studentas]")

6.1.1 Macro Definition Documentation

6.1.1.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

6.1.2 Function Documentation

6.1.2.1 TEST_CASE()

```
TEST_CASE (
    "Studento klasės 'Rule of five' testai" ,
    "[studentas]")
```

6.2 funkcijos.cpp File Reference

```
#include "studentas.h"
```

Functions

- void [failu_generavimas](#) (int *Studentu_sk*)
- void [studentu_skirstymas](#) (vector< [Studentas](#) > &grupe)
- void [isvedimas_i_du_failus](#) (vector< [Studentas](#) > &vargsiukai, vector< [Studentas](#) > &kietakai)
- void [skaiciuoti_viska](#) ([Studentas](#) &x)
- void [skaityti_faila_automatiskai](#) (string pav, vector< [Studentas](#) > &grupe)
- string [pasirinkimas](#) ()
- void [skaityti_faila](#) (vector< [Studentas](#) > &grupe)
- void [duomenu_ivedimas](#) (vector< [Studentas](#) > &grupe)
- void [vidurkis](#) ([Studentas](#) &x)
- void [mediana](#) ([Studentas](#) &x)
- void [skaiciu_generevimas](#) ([Studentas](#) &A)
- void [rezultatai](#) (vector< [Studentas](#) > &grupe)
- void [rezultatu_isvedimas](#) (vector< [Studentas](#) > &grupe)
- bool [pagal_varda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_pavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_vid_did](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_vid_maz](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_med_did](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_med_maz](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- void [rikiavimas](#) (vector< [Studentas](#) > &grupe)
- void [menu](#) (vector< [Studentas](#) > &grupe)
- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &A)

6.2.1 Function Documentation

6.2.1.1 duomenu_ivedimas()

```
void duomenu_ivedimas (
    vector< Studentas > & grupe)
```

6.2.1.2 failu_generavimas()

```
void failu_generavimas (
    int Studentu_sk)
```

6.2.1.3 isvedimas_i_du_failus()

```
void isvedimas_i_du_failus (
    vector< Studentas > & vargsiukai,
    vector< Studentas > & kietakai)
```

6.2.1.4 mediana()

```
void mediana (  
    Studentas & x)
```

6.2.1.5 meniu()

```
void meniu (  
    vector< Studentas > & grupe)
```

6.2.1.6 operator<<()

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Studentas & A)
```

6.2.1.7 operator>>()

```
std::istream & operator>> (  
    std::istream & in,  
    Studentas & s)
```

6.2.1.8 pagal_med_did()

```
bool pagal_med_did (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.9 pagal_med_maz()

```
bool pagal_med_maz (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.10 pagal_pavarde()

```
bool pagal_pavarde (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.11 pagal_varda()

```
bool pagal_varda (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.12 pagal_vid_did()

```
bool pagal_vid_did (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.13 pagal_vid_maz()

```
bool pagal_vid_maz (  
    const Studentas & a,  
    const Studentas & b)
```

6.2.1.14 pasirinkimas()

```
string pasirinkimas ()
```

6.2.1.15 rezultatai()

```
void rezultatai (  
    vector< Studentas > & grupe)
```

6.2.1.16 rezultatu_isvedimas()

```
void rezultatu_isvedimas (  
    vector< Studentas > & grupe)
```

6.2.1.17 rikiavimas()

```
void rikiavimas (  
    vector< Studentas > & grupe)
```

6.2.1.18 skaiciu_generevimas()

```
void skaiciu_generevimas (  
    Studentas & A)
```

6.2.1.19 skaiciuoti_viska()

```
void skaiciuoti_viska (  
    Studentas & x)
```

6.2.1.20 skaityti_faila()

```
void skaityti_faila (  
    vector< Studentas > & grupe)
```

6.2.1.21 skaityti_faila_automatiskai()

```
void skaityti_faila_automatiskai (
    string pav,
    vector< Studentas > & grupe)
```

6.2.1.22 studentu_skirstymas()

```
void studentu_skirstymas (
    vector< Studentas > & grupe)
```

6.2.1.23 vidurkis()

```
void vidurkis (
    Studentas & x)
```

6.3 konteineriai.h File Reference

```
#include "studentas.h"
#include <list>
#include <type_traits>
#include <algorithm>
```

Functions

- template<typename Container>
void [skaityti_failus](#) (string pav, Container &grupe)
- template<typename Container>
void [rusiavimas_maz](#) (Container &grupe)
- template<typename Container>
void [stunedu_skirstymas](#) (Container &grupe)
- template<typename Container>
void [pirma_strategija](#) (Container &grupe)
- template<typename Container>
void [antra_strategija](#) (Container &grupe)
- template<typename Container>
void [trecia_strategija](#) (Container &grupe)
- template<typename Container>
void [tyrimas](#) (string failas, Container &grupe, string tipas)
- template<typename Container>
void [strategiju_tyrimas](#) (string failas, Container &grupe, string tipas)
- template<typename Container>
void [strategijos_du_tyrimas](#) (string failas, Container &grupe, string tipas)
- template<typename Container>
void [strategijos_trys_tyrimas](#) (string failas, Container &grupe, string tipas)
- template<typename Container>
void [isvedimas](#) (Container &vargsiukai, Container &kietakai)

6.3.1 Function Documentation

6.3.1.1 antra_strategija()

```
template<typename Container>
void antra_strategija (
    Container & grupe)
```

6.3.1.2 isvedimas()

```
template<typename Container>
void isvedimas (
    Container & vargsiukai,
    Container & kietakai)
```

6.3.1.3 pirma_strategija()

```
template<typename Container>
void pirma_strategija (
    Container & grupe)
```

6.3.1.4 rusiavimas_maz()

```
template<typename Container>
void rusiavimas_maz (
    Container & grupe)
```

6.3.1.5 skaityti_failus()

```
template<typename Container>
void skaityti_failus (
    string pav,
    Container & grupe)
```

6.3.1.6 strategijos_du_tyrimas()

```
template<typename Container>
void strategijos_du_tyrimas (
    string failas,
    Container & grupe,
    string tipas)
```

6.3.1.7 strategijos_trys_tyrimas()

```
template<typename Container>
void strategijos_trys_tyrimas (
    string failas,
    Container & grupe,
    string tipas)
```

6.3.1.8 strategiju_tyrimas()

```
template<typename Container>
void strategiju_tyrimas (
    string failas,
    Container & grupe,
    string tipas)
```

6.3.1.9 stunedu_skirstymas()

```
template<typename Container>
void stunedu_skirstymas (
    Container & grupe)
```

6.3.1.10 trecia_strategija()

```
template<typename Container>
void trecia_strategija (
    Container & grupe)
```

6.3.1.11 tyrimas()

```
template<typename Container>
void tyrimas (
    string failas,
    Container & grupe,
    string tipas)
```

6.4 konteineriai.h

[Go to the documentation of this file.](#)

```
00001 #ifndef KONTEINERIAI_H
00002 #define KONTEINERIAI_H
00003 #include "studentas.h"
00004 #include <list>
00005 #include <type_traits>
00006 #include <algorithm>
00007
00008 using std:: deque;
00009 using std:: list;
00010
00011 template < typename Container >
00012 void skaityti_failus (string pav, Container& grupe)
00013 {
00014     string v, p;
00015
00016     std::ifstream fd(pav);
00017     if (!fd)
00018     {
00019         std::cerr << "Klaida: nepavyko atidaryti failo!" << endl;
00020         return;
00021     }
00022
00023     string eil;
00024     getline(fd, eil);
00025
00026     while(getline(fd, eil))
00027     {
00028         if(eil.empty()) continue;
```

```

00029         Studentas A;
00030         std::istream eilute(eil);
00031         if (eilute » A)
00032         {
00033             skaiciuoti_viska(A);
00034             grupe.push_back(std::move(A));
00035         }
00036     }
00037     fd.close();
00038 }
00039
00040
00041 template < typename Container >
00042 void rusiavimas_maz (Container& grupe)
00043 {
00044     if constexpr (std::is_same_v<Container, std::list<Studentas> >)
00045     {
00046         grupe.sort([](const Studentas &a, const Studentas &b)
00047         {
00048             return a.vidurkis() > b.vidurkis();
00049         });
00050     }
00051     else
00052     {
00053         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b)
00054         {
00055             return a.vidurkis() > b.vidurkis();
00056         });
00057     }
00058 }
00059 }
00060
00061 template < typename Container >
00062 void stunedu_skirstymas (Container& grupe)
00063 {
00064     Container vargsiukai;
00065     Container kietakai;
00066
00067     if constexpr (std::is_same_v<Container, std::vector<Studentas> >)
00068     {
00069         vargsiukai.reserve(grupe.size() / 2);
00070         kietakai.reserve(grupe.size() / 2);
00071     }
00072
00073     for (auto &x : grupe)
00074     {
00075         if (x.vidurkis() < 5.0)
00076             vargsiukai.push_back(std::move(x));
00077         else
00078             kietakai.push_back(std::move(x));
00079     }
00080
00081     isvedimas(vargsiukai, kietakai);
00082
00083     grupe.clear();
00084 }
00085
00086
00087 template <typename Container>
00088 void pirma_strategija(Container& grupe){
00089     Container vargsiukai;
00090     Container kietakai;
00091
00092     for (const auto &s : grupe)
00093     {
00094         if (s.vidurkis() < 5){
00095             vargsiukai.push_back(s);
00096         } else if (s.vidurkis() >= 5){
00097             kietakai.push_back(s);
00098         }
00099     }
00100
00101     if constexpr (std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
00102 deque<Studentas> >){
00103         vargsiukai.shrink_to_fit();
00104         kietakai.shrink_to_fit();
00105     }
00106
00107     isvedimas(vargsiukai, kietakai);
00108 }
00109
00110 template <typename Container>
00111 void antra_strategija (Container& grupe)
00112 {
00113     Container vargsiukai;
00114     while (!grupe.empty() && grupe.back().vidurkis() < 5.0)
00115     {

```

```

00115         vargsiukai.push_back(std::move(grupe.back()));
00116         grupe.pop_back();
00117     }
00118
00119     isvedimas(vargsiukai, grupe);
00120 }
00121
00122 template <typename Container>
00123 void trecia_strategija(Container& grupe) {
00124     Container vargsiukai;
00125
00126     auto it = std::stable_partition(grupe.begin(), grupe.end(), [](const Studentas& s)
00127     {
00128         return s.vidurkis() >= 5.0;
00129     });
00130
00131     std::move(it, grupe.end(), std::back_inserter(vargsiukai));
00132     grupe.erase(it, grupe.end());
00133
00134     if constexpr(std::is_same_v<Container, std::vector<Studentas>>)
00135     {
00136         grupe.shrink_to_fit();
00137     }
00138
00139     isvedimas(vargsiukai, grupe);
00140 }
00141
00142 template <typename Container>
00143 void tyrimas(string failas, Container& grupe, string tipas)
00144 {
00145     double nuskaitym = 0, rikiav = 0, skirst = 0;
00146
00147     cout << "Testuojamas " << tipas << endl;
00148
00149     for (int i = 0; i < 3; i++)
00150     {
00151         grupe.clear();
00152         auto s = std::chrono::high_resolution_clock::now();
00153         skaityti_failus(failas, grupe);
00154         auto e = std::chrono::high_resolution_clock::now();
00155         nuskaitym += std::chrono::duration<double>(e - s).count();
00156
00157         s = std::chrono::high_resolution_clock::now();
00158         rusiavimas_maz(grupe);
00159         e = std::chrono::high_resolution_clock::now();
00160         rikiav += std::chrono::duration<double>(e - s).count();
00161
00162         s = std::chrono::high_resolution_clock::now();
00163         stunedu_skirstymas(grupe);
00164         e = std::chrono::high_resolution_clock::now();
00165         skirst += std::chrono::duration<double>(e - s).count();
00166     }
00167
00168     cout << tipas << " rezultatai (vidurkis is 3 bandymu):" << endl;
00169     cout << " Nuskaitymas užtruko: " << nuskaitym / 3.0 << " s" << endl;
00170     cout << " Rikiavimas užtruko: " << rikiav / 3.0 << " s" << endl;
00171     cout << " Skirstymas užtruko: " << skirst / 3.0 << " s" << endl;
00172     cout << " Bendras veikimos laikas: " << (nuskaitym + rikiav + skirst) / 3.0 << " s" << endl;
00173 }
00174 }
00175
00176 template <typename Container>
00177 void strategiju_tyrimas(string failas, Container& grupe, string tipas)
00178 {
00179     double nuskaitym = 0, rikiav = 0, skirst = 0;
00180
00181
00182
00183     for (int i = 0; i < 3; i++)
00184     {
00185         grupe.clear();
00186         auto s = std::chrono::high_resolution_clock::now();
00187         skaityti_failus(failas, grupe);
00188         auto e = std::chrono::high_resolution_clock::now();
00189         nuskaitym += std::chrono::duration<double>(e - s).count();
00190
00191         s = std::chrono::high_resolution_clock::now();
00192         rusiavimas_maz(grupe);
00193         e = std::chrono::high_resolution_clock::now();
00194         rikiav += std::chrono::duration<double>(e - s).count();
00195
00196         s = std::chrono::high_resolution_clock::now();
00197         pirma_strategija(grupe);
00198         e = std::chrono::high_resolution_clock::now();
00199         skirst += std::chrono::duration<double>(e - s).count();
00200
00201     }

```

```

00202     cout << "Programa su " << tipas << " vidutiniškai užtruko:      " << (nuskaitym + rikiav + skirst) /
3.0 << " s" << endl;
00203 }
00204
00205 template <typename Container>
00206 void strategijos_du_tyrimas (string failas, Container& grupe, string tipas)
00207 {
00208     double nuskaitym = 0, rikiav = 0, skirst = 0;
00209
00210
00211
00212     for (int i = 0; i < 3; i++)
00213     {
00214         grupe.clear();
00215         auto s = std::chrono::high_resolution_clock::now();
00216         skaityti_failus(failas, grupe);
00217         auto e = std::chrono::high_resolution_clock::now();
00218         nuskaitym += std::chrono::duration<double>(e - s).count();
00219
00220         s = std::chrono::high_resolution_clock::now();
00221         rusiavimas_maz(grupe);
00222         e = std::chrono::high_resolution_clock::now();
00223         rikiav += std::chrono::duration<double>(e - s).count();
00224
00225         s = std::chrono::high_resolution_clock::now();
00226         antra_strategija(grupe);
00227         e = std::chrono::high_resolution_clock::now();
00228         skirst += std::chrono::duration<double>(e - s).count();
00229     }
00230
00231     cout << "Programa su " << tipas << " vidutiniškai užtruko:      " << (nuskaitym + rikiav + skirst) /
3.0 << " s" << endl;
00232 }
00233
00234 template <typename Container>
00235 void strategijos_trys_tyrimas (string failas, Container& grupe, string tipas)
00236 {
00237     double nuskaitym = 0, rikiav = 0, skirst = 0;
00238
00239
00240
00241     for (int i = 0; i < 3; i++)
00242     {
00243         grupe.clear();
00244         auto s = std::chrono::high_resolution_clock::now();
00245         skaityti_failus(failas, grupe);
00246         auto e = std::chrono::high_resolution_clock::now();
00247         nuskaitym += std::chrono::duration<double>(e - s).count();
00248
00249         s = std::chrono::high_resolution_clock::now();
00250         rusiavimas_maz(grupe);
00251         e = std::chrono::high_resolution_clock::now();
00252         rikiav += std::chrono::duration<double>(e - s).count();
00253
00254         s = std::chrono::high_resolution_clock::now();
00255         trecia_strategija(grupe);
00256         e = std::chrono::high_resolution_clock::now();
00257         skirst += std::chrono::duration<double>(e - s).count();
00258     }
00259
00260     cout << "Programa su " << tipas << " vidutiniškai užtruko:      " << (nuskaitym + rikiav + skirst) /
3.0 << " s" << endl;
00261 }
00262
00263 template < typename Container >
00264 void isvedimas (Container &vargsiukai, Container &kietakai)
00265 {
00266
00267     auto i_faila = [] (string pav, Container& duomenys)
00268     {
00269         std::ofstream fr(pav);
00270         fr << left << setw(20) << "Vardas" << setw(20) << "Pavarde" << setw(20) << "Galutinis" << endl;
00271         for (const auto &s : duomenys) {
00272             fr << left << setw(20) << s.vardas() << setw(20) << s.pavarde() << std::fixed <<
std::setprecision(2) << s.vidurkis() << "\n";
00273         }
00274         fr.close();
00275     };
00276
00277     i_faila("vargsiukai.txt", vargsiukai);
00278
00279     i_faila("kietakai.txt", kietakai);
00280 }
00281 }
00282
00283 #endif

```

6.5 kontainerius_tyrimas.cpp File Reference

```
#include "studentas.h"
#include "kontaineriai.h"
```

Functions

- int [main](#) ()

6.5.1 Function Documentation

6.5.1.1 main()

```
int main ()
```

6.6 RandInt.hpp File Reference

```
#include <random>
```

Classes

- class [RandInt](#)

6.7 RandInt.hpp

[Go to the documentation of this file.](#)

```
00001 #include <random>
00002
00003 class RandInt {
00004     public:
00005         RandInt(int low, int high) : mt{rd()}, dist{low, high} { }
00006         int operator() () { return dist(mt); } // generating of Int values
00007     private:
00008         std::random_device rd;
00009         std::mt19937 mt;
00010         std::uniform_int_distribution<int> dist;
00011 };
00012
00013 #pragma once
```

6.8 README.md File Reference

6.9 strategija_1.cpp File Reference

```
#include "studentas.h"
#include "kontaineriai.h"
```

Functions

- int `main` ()

6.9.1 Function Documentation

6.9.1.1 main()

```
int main ()
```

6.10 strategija_2.cpp File Reference

```
#include "studentas.h"  
#include "konteineriai.h"
```

Functions

- int `main` ()

6.10.1 Function Documentation

6.10.1.1 main()

```
int main ()
```

6.11 strategija_3.cpp File Reference

```
#include "studentas.h"  
#include "konteineriai.h"
```

Functions

- int `main` ()

6.11.1 Function Documentation

6.11.1.1 main()

```
int main ()
```

6.12 studentas.h File Reference

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <numeric>
#include <algorithm>
#include <limits>
#include <random>
#include "RandInt.hpp"
#include <fstream>
#include <sstream>
#include <chrono>
#include "zmogus.h"
```

Classes

- class [Studentas](#)

Functions

- void [failu_generavimas](#) (int Studentu_sk)
- void [studentu_skirstymas](#) (vector< [Studentas](#) > &grupe)
- void [isvedimas_i_du_failus](#) (vector< [Studentas](#) > &vargsiukai, vector< [Studentas](#) > &kietakai)
- void [skaiciuoti_viska](#) ([Studentas](#) &x)
- void [skaityti_faila_automatiskai](#) (string pav, vector< [Studentas](#) > &grupe)
- string [pasirinkimas](#) ()
- void [skaityti_faila](#) (vector< [Studentas](#) > &grupe)
- void [duomenu_ivedimas](#) (vector< [Studentas](#) > &grupe)
- void [skaiciu_generevimas](#) ([Studentas](#) &A)
- void [rezultatai](#) (vector< [Studentas](#) > &grupe)
- void [rezultatu_isvedimas](#) (vector< [Studentas](#) > &grupe)
- bool [pagal_varda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_pavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_vid_did](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_vid_maz](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_med_did](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [pagal_med_maz](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- void [rikiavimas](#) (vector< [Studentas](#) > &grupe)
- void [menu](#) (vector< [Studentas](#) > &grupe)

6.12.1 Function Documentation

6.12.1.1 duomenu_ivedimas()

```
void duomenu_ivedimas (
    vector< Studentas > & grupe)
```

6.12.1.2 failu_generavimas()

```
void failu_generavimas (  
    int Studentu_sk)
```

6.12.1.3 isvedimas_i_du_failus()

```
void isvedimas_i_du_failus (  
    vector< Studentas > & vargsiukai,  
    vector< Studentas > & kietakai)
```

6.12.1.4 meniu()

```
void meniu (  
    vector< Studentas > & grupe)
```

6.12.1.5 pagal_med_did()

```
bool pagal_med_did (  
    const Studentas & a,  
    const Studentas & b)
```

6.12.1.6 pagal_med_maz()

```
bool pagal_med_maz (  
    const Studentas & a,  
    const Studentas & b)
```

6.12.1.7 pagal_pavarde()

```
bool pagal_pavarde (  
    const Studentas & a,  
    const Studentas & b)
```

6.12.1.8 pagal_varda()

```
bool pagal_varda (  
    const Studentas & a,  
    const Studentas & b)
```

6.12.1.9 pagal_vid_did()

```
bool pagal_vid_did (  
    const Studentas & a,  
    const Studentas & b)
```

6.12.1.10 pagal_vid_maz()

```
bool pagal_vid_maz (
    const Studentas & a,
    const Studentas & b)
```

6.12.1.11 pasirinkimas()

```
string pasirinkimas ()
```

6.12.1.12 rezultatai()

```
void rezultatai (
    vector< Studentas > & grupe)
```

6.12.1.13 rezultatu_isvedimas()

```
void rezultatu_isvedimas (
    vector< Studentas > & grupe)
```

6.12.1.14 rikiavimas()

```
void rikiavimas (
    vector< Studentas > & grupe)
```

6.12.1.15 skaiciu_generevimas()

```
void skaiciu_generevimas (
    Studentas & A)
```

6.12.1.16 skaiciuoti_viska()

```
void skaiciuoti_viska (
    Studentas & x)
```

6.12.1.17 skaityti_faila()

```
void skaityti_faila (
    vector< Studentas > & grupe)
```

6.12.1.18 skaityti_faila_automatiskai()

```
void skaityti_faila_automatiskai (
    string pav,
    vector< Studentas > & grupe)
```

6.12.1.19 studentu_skirstymas()

```
void studentu_skirstymas (
    vector< Studentas > & grupe)
```

6.13 studentas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <string>
00007 #include <vector>
00008 #include <numeric>
00009 #include <algorithm>
00010 #include <limits>
00011 #include <random>
00012 #include "RandInt.hpp"
00013 #include <fstream>
00014 #include <sstream>
00015 #include <chrono>
00016 #include "zmogus.h"
00017
00018 using std:: string;
00019 using std:: vector;
00020 using std:: cout;
00021 using std:: cin;
00022 using std:: accumulate;
00023 using std:: setw;
00024 using std:: left;
00025 using std:: right;
00026 using std:: endl;
00027
00028
00029 class Studentas : public Zmogus {
00030     private:
00031         vector<int> paz_;
00032         int egz_paz_;
00033         double Vidurkis_;
00034         double Mediana_;
00035     public:
00036
00037         Studentas() : Zmogus(), paz_({0}), egz_paz_(0), Vidurkis_(0.0), Mediana_(0.0) { }
00038
00039         Studentas(string v, string p, vector<int> pazymiai) : Zmogus(v, p), paz_(pazymiai),
00040             egz_paz_(0), Vidurkis_(0.0), Mediana_(0.0) { }
00041
00042         Studentas(std::istream& is);
00043
00044         // copy konstruktorius
00045         Studentas(const Studentas& s):
00046             Zmogus(s),
00047             paz_(s.paz_),
00048             egz_paz_(s.egz_paz_),
00049             Vidurkis_(s.Vidurkis_),
00050             Mediana_(s.Mediana_) {}
00051
00052         // move konstruktorius
00053         Studentas(Studentas&& s) noexcept :
00054             Zmogus(std::move(s)),
00055             paz_(std::move(s.paz_)),
00056             egz_paz_(s.egz_paz_),
00057             Vidurkis_(s.Vidurkis_),
00058             Mediana_(s.Mediana_) { s.egz_paz_ = 0; s.Vidurkis_ = 0.0; s.Mediana_ = 0.0; s.Clear();}
00059
00060         ~Studentas() {
00061             Clear();
00062         }
00063
00064         inline const vector<int>& pazymiai() const { return paz_; }
00065         inline int egzaminas() const { return egz_paz_; }
00066         inline double vidurkis() const { return Vidurkis_; }
00067         inline double mediana() const { return Mediana_; }
00068
00069 }
```

```

00070     inline void setGalutinisV(double galutVid) { Vidurkis_ = galutVid; }
00071     inline void setGalutinisM(double galutMed) { Mediana_ = galutMed; }
00072     inline void setEgzaminas(int egz) { egz_paz_ = egz; }
00073     inline void setPazymiai(vector<int> paz) { paz_ = paz; }
00074
00075     void apskaiciuoti_vidurkis();
00076     void apskaiciuoti_mediana();
00077
00078     // copy assignment operatorius
00079     Studentas& operator = (const Studentas& s)
00080     {
00081         if (this == &s)
00082             return *this;
00083
00084         Zmogus::operator=(s);
00085         paz_ = s.paz_;
00086         egz_paz_ = s.egz_paz_;
00087         Vidurkis_ = s.Vidurkis_;
00088         Mediana_ = s.Mediana_;
00089
00090         return *this;
00091     }
00092
00093     // move assignment operatorius
00094     Studentas& operator = (Studentas&& s) noexcept
00095     {
00096         if (this == &s)
00097             return *this;
00098
00099         Zmogus::operator=(std::move(s));
00100         paz_ = std::move(s.paz_);
00101         egz_paz_ = s.egz_paz_;
00102         Vidurkis_ = s.Vidurkis_;
00103         Mediana_ = s.Mediana_;
00104
00105         s.Clear();
00106
00107         return *this;
00108     }
00109
00110     friend bool operator==(const Studentas& s1, const Studentas& s2)
00111     {
00112         return (s1.Vardas_ == s2.Vardas_ && s1.Pavarde_ == s2.Pavarde_ && s1.egz_paz_ ==
00113 s2.egz_paz_ && s1.Vidurkis_ == s2.Vidurkis_ && s1.Mediana_ == s2.Mediana_);
00114     }
00115
00116     //input operatorius
00117     friend std::istream& operator>(std::istream& in, Studentas& s);
00118
00119     //output operatorius
00120     friend std::ostream& operator<(std::ostream& out, const Studentas& s);
00121
00122     void Clear()
00123     {
00124         Vardas_.clear();
00125         Pavarde_.clear();
00126         paz_.clear();
00127         egz_paz_ = 0;
00128         Vidurkis_ = 0.0;
00129         Mediana_ = 0.0;
00130     }
00131
00132     bool isClear(){
00133         if(Vardas_.empty() && Pavarde_.empty() && paz_.empty() && egz_paz_ == 0 && Vidurkis_ ==
00134 0.0 && Mediana_ == 0.0) {
00135             return true;
00136         } else {
00137             return false;
00138         }
00139     }
00140
00141     void printInfo() const override
00142     {
00143         cout << "Vardas: " << vardas() << ", Pavarde: " << pavarde() << endl;
00144     }
00145
00146 void failu_generavimas(int Studentu_sk);
00147 void studentu_skirstymas(vector < Studentas > &grupe);
00148 void isvedimas_i_du_failus (vector < Studentas > &vargsiukai, vector < Studentas > &kietakai);
00149 void skaiciuoti_viska(Studentas &x);
00150 void skaityti_faile_automatiskai(string pav, vector<Studentas>& grupe);
00151 string pasirinkimas ();
00152 void skaityti_faile(vector < Studentas > &grupe);
00153 void duomenu_ivedimas(vector < Studentas > &grupe);

```

```
00155 void skaiciu_generevimas(Studentas &A);
00156 void rezultatai (vector < Studentas > &grupe);
00157 void rezultatu_isvedimas(vector < Studentas > &grupe);
00158 bool pagal_varda(const Studentas &a, const Studentas &b);
00159 bool pagal_pavarde(const Studentas &a, const Studentas &b);
00160 bool pagal_vid_did(const Studentas &a, const Studentas &b);
00161 bool pagal_vid_maz(const Studentas &a, const Studentas &b);
00162 bool pagal_med_did(const Studentas &a, const Studentas &b);
00163 bool pagal_med_maz(const Studentas &a, const Studentas &b);
00164 void rikiavimas(vector < Studentas > &grupe);
00165 void meniu(vector < Studentas > &grupe);
00166
00167
00168 #endif
```

6.14 vector.cpp File Reference

```
#include "studentas.h"
```

Functions

- int [main](#) ()

6.14.1 Function Documentation

6.14.1.1 main()

```
int main ()
```

6.15 zmogus.h File Reference

```
#include <iostream>
#include <utility>
```

Classes

- class [Zmogus](#)

6.16 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include <utility>
00003 using std::string;
00004
00005 class Zmogus {
00006     protected:
00007         string Vardas_;
00008         string Pavarde_;
00009     public:
00010         Zmogus() : Vardas_(""), Pavarde_("") {}
00011         Zmogus(string v, string p) : Vardas_(v), Pavarde_(p) {}
00012
00013         Zmogus(const Zmogus& z):
00014             Vardas_(z.Vardas_),
00015             Pavarde_(z.Pavarde_) {}
00016
00017         Zmogus(Zmogus&& z) noexcept
00018             : Vardas_(std::move(z.Vardas_)),
00019             Pavarde_(std::move(z.Pavarde_)) {}
00020
00021
00022         inline string vardas() const { return Vardas_; }
00023         inline string pavarde() const { return Pavarde_; }
00024
00025         inline void setVardas(string var) { Vardas_ = var; }
00026         inline void setPavarde(string pav) { Pavarde_ = pav; }
00027
00028         Zmogus& operator=(const Zmogus& z)
00029         {
00030             if (this != &z) {
00031                 Vardas_ = z.Vardas_;
00032                 Pavarde_ = z.Pavarde_;
00033             }
00034             return *this;
00035         }
00036
00037         Zmogus& operator=(Zmogus&& z) noexcept
00038         {
00039             if (this != &z) {
00040                 Vardas_ = std::move(z.Vardas_);
00041                 Pavarde_ = std::move(z.Pavarde_);
00042             }
00043             return *this;
00044         }
00045
00046         virtual ~Zmogus() {
00047             Vardas_.clear();
00048             Pavarde_.clear();
00049         }
00050
00051         virtual void printInfo() const = 0;
00052 };

```


Index

~Studentas
 Studentas, [14](#)
~Zmogus
 Zmogus, [18](#)

antra_strategija
 konteineriai.h, [26](#)
Antras_Laboratorinis, [1](#)
apskaiciuot_vidurkis
 Studentas, [14](#)
apskaiciuoti_mediana
 Studentas, [14](#)

CATCH_CONFIG_MAIN
 catch_testai.cpp, [21](#)
catch_testai.cpp, [21](#)
 CATCH_CONFIG_MAIN, [21](#)
 TEST_CASE, [21](#)
Clear
 Studentas, [14](#)

dist
 RandInt, [12](#)
duomenu_ivedimas
 funkcijos.cpp, [22](#)
 studentas.h, [33](#)

egz_paz_
 Studentas, [16](#)
egzaminas
 Studentas, [14](#)

failu_generavimas
 funkcijos.cpp, [22](#)
 studentas.h, [33](#)
funkcijos.cpp, [22](#)
 duomenu_ivedimas, [22](#)
 failu_generavimas, [22](#)
 isvedimas_i_du_failus, [22](#)
 mediana, [22](#)
 menu, [23](#)
 operator<<, [23](#)
 operator>>, [23](#)
 pagal_med_did, [23](#)
 pagal_med_maz, [23](#)
 pagal_pavarde, [23](#)
 pagal_varda, [23](#)
 pagal_vid_did, [23](#)
 pagal_vid_maz, [24](#)
 pasirinkimas, [24](#)
 rezultatai, [24](#)

 rezultatu_isvedimas, [24](#)
 rikiavimas, [24](#)
 skaiciu_generevimas, [24](#)
 skaiciuoti_viska, [24](#)
 skaityti_faila, [24](#)
 skaityti_faila_automatiskai, [24](#)
 studentu_skirstymas, [25](#)
 vidurkis, [25](#)

isClear
 Studentas, [14](#)
isvedimas
 konteineriai.h, [26](#)
isvedimas_i_du_failus
 funkcijos.cpp, [22](#)
 studentas.h, [34](#)

konteineriai.h, [25](#)
 antra_strategija, [26](#)
 isvedimas, [26](#)
 pirma_strategija, [26](#)
 rusiavimas_maz, [26](#)
 skaityti_failus, [26](#)
 strategijos_du_tyrimas, [26](#)
 strategijos_trys_tyrimas, [26](#)
 strategiju_tyrimas, [26](#)
 stunedu_skirstymas, [27](#)
 trecia_strategija, [27](#)
 tyrimas, [27](#)
konteinerius_tyrimas.cpp, [31](#)
 main, [31](#)

main
 konteinerius_tyrimas.cpp, [31](#)
 strategija_1.cpp, [32](#)
 strategija_2.cpp, [32](#)
 strategija_3.cpp, [32](#)
 vector.cpp, [38](#)
mediana
 funkcijos.cpp, [22](#)
 Studentas, [14](#)
Mediana_
 Studentas, [16](#)
menu
 funkcijos.cpp, [23](#)
 studentas.h, [34](#)
mt
 RandInt, [12](#)

operator<<

- funkcijos.cpp, 23
 - Studentas, 16
- operator>>
 - funkcijos.cpp, 23
 - Studentas, 16
- operator()
 - RandInt, 11
- operator=
 - Studentas, 15
 - Zmogus, 18
- operator==
 - Studentas, 16
- pagal_med_did
 - funkcijos.cpp, 23
 - studentas.h, 34
- pagal_med_maz
 - funkcijos.cpp, 23
 - studentas.h, 34
- pagal_pavarde
 - funkcijos.cpp, 23
 - studentas.h, 34
- pagal_varda
 - funkcijos.cpp, 23
 - studentas.h, 34
- pagal_vid_did
 - funkcijos.cpp, 23
 - studentas.h, 34
- pagal_vid_maz
 - funkcijos.cpp, 24
 - studentas.h, 34
- pasirinkimas
 - funkcijos.cpp, 24
 - studentas.h, 35
- pavarde
 - Zmogus, 18
- Pavarde_
 - Zmogus, 19
- paz_
 - Studentas, 16
- pazymiai
 - Studentas, 15
- pirma_strategija
 - konteineriai.h, 26
- printlnfo
 - Studentas, 15
 - Zmogus, 18
- RandInt, 11
 - dist, 12
 - mt, 12
 - operator(), 11
 - RandInt, 11
 - rd, 12
- RandInt.hpp, 31
- rd
 - RandInt, 12
- README.md, 31
- rezultatai
 - funkcijos.cpp, 24
 - studentas.h, 35
- rezultatu_isvedimas
 - funkcijos.cpp, 24
 - studentas.h, 35
- rikiavimas
 - funkcijos.cpp, 24
 - studentas.h, 35
- rusiavimas_maz
 - konteineriai.h, 26
- setEgzaminas
 - Studentas, 15
- setGalutinisM
 - Studentas, 15
- setGalutinisV
 - Studentas, 15
- setPavarde
 - Zmogus, 18
- setPazymiai
 - Studentas, 15
- setVardas
 - Zmogus, 18
- skaiciu_generevimas
 - funkcijos.cpp, 24
 - studentas.h, 35
- skaiciuoti_viska
 - funkcijos.cpp, 24
 - studentas.h, 35
- skaityti_faila
 - funkcijos.cpp, 24
 - studentas.h, 35
- skaityti_faila_automatiskai
 - funkcijos.cpp, 24
 - studentas.h, 35
- skaityti_failus
 - konteineriai.h, 26
- strategija_1.cpp, 31
 - main, 32
- strategija_2.cpp, 32
 - main, 32
- strategija_3.cpp, 32
 - main, 32
- strategijos_du_tyrimas
 - konteineriai.h, 26
- strategijos_trys_tyrimas
 - konteineriai.h, 26
- strategiju_tyrimas
 - konteineriai.h, 26
- Studentas, 12
 - ~Studentas, 14
 - apskaiciuot_vidurkis, 14
 - apskaiciuoti_mediana, 14
 - Clear, 14
 - egz_paz_, 16
 - egzaminas, 14
 - isClear, 14
 - mediana, 14
 - Mediana_, 16

- operator<<, 16
- operator>>, 16
- operator=, 15
- operator==, 16
- paz_, 16
- pazymiai, 15
- printInfo, 15
- setEgzaminas, 15
- setGalutinisM, 15
- setGalutinisV, 15
- setPazymiai, 15
- Studentas, 13, 14
- vidurkis, 15
- Vidurkis_, 16
- studentas.h, 33
 - duomenu_ivedimas, 33
 - failu_generavimas, 33
 - isvedimas_i_du_failus, 34
 - menu, 34
 - pagal_med_did, 34
 - pagal_med_maz, 34
 - pagal_pavarde, 34
 - pagal_varda, 34
 - pagal_vid_did, 34
 - pagal_vid_maz, 34
 - pasirinkimas, 35
 - rezultatai, 35
 - rezultatu_isvedimas, 35
 - rikiavimas, 35
 - skaiciu_generevimas, 35
 - skaiciuoti_viska, 35
 - skaityti_faila, 35
 - skaityti_faila_automatiskai, 35
 - studentu_skirstymas, 35
- studentu_skirstymas
 - funkcijos.cpp, 25
 - studentas.h, 35
- stunedu_skirstymas
 - konteineriai.h, 27
- TEST_CASE
 - catch_testai.cpp, 21
- trecia_strategija
 - konteineriai.h, 27
- tyrimas
 - konteineriai.h, 27
- vardas
 - Zmogus, 18
- Vardas_
 - Zmogus, 19
- vector.cpp, 38
 - main, 38
- vidurkis
 - funkcijos.cpp, 25
 - Studentas, 15
- Vidurkis_
 - Studentas, 16
- Zmogus, 17
 - ~Zmogus, 18
 - operator=, 18
 - pavarde, 18
 - Pavarde_, 19
 - printInfo, 18
 - setPavarde, 18
 - setVardas, 18
 - vardas, 18
 - Vardas_, 19
 - Zmogus, 17
- zmogus.h, 38